

CSE420 – Compiler Design
Study Notes

Construction of SLR Parsing Table

and the SLR(1) Parsing Algorithm

Prepared by
Azmari Sultana
BSc in Computer Science and Engineering
BRAC University

Table of Contents

1. Introduction — Why We Need LR Parsing.....	3
2. Structure of the LR Parsing Table.....	4
3. Behavior of the LR Parser — the Four Moves.....	6
4. Worked Example — LR Parsing Table for the Expression Grammar.....	8
5. Constructing SLR(1) Parsing Tables.....	11
6. Viable Prefixes — Why the LR(0) Automaton Is Trustworthy.....	17
7. FIRST and FOLLOW Sets — the Ingredients Behind the Tables.....	19
8. Using Ambiguous Grammars — the Dangling-Else Trace.....	21
9. Error Recovery in LR Parsing.....	22
10. The Mechanics of Every Single Move — Slow-Motion Simulation.....	26
11. Fully Worked Practice Problems.....	29

1. Introduction — Why We Need LR Parsing

Before jumping into SLR tables, let us remind ourselves what problem we are solving. A **parser** is the part of a compiler that checks whether a stream of tokens (produced by the lexical analyzer) actually follows the grammar rules of the programming language, and if so, builds a parse tree (or at least the sequence of reductions) that shows how the input was derived from the start symbol.

LR parsing is a **bottom-up** parsing technique. "L" means we scan the input from Left to right, and "R" means we construct a Rightmost derivation, but in reverse (that is, we discover the rightmost derivation steps in reverse order as we read the input left to right). LR parsers are attractive because:

- They can be built for a very large class of context-free grammars (much larger than LL parsers can handle).
- They detect a syntax error as soon as possible — as soon as no valid continuation of the input exists.
- There exist well known table-construction methods: LR(0), SLR(1), Canonical LR(1), and LALR(1). This note focuses on SLR(1), the Simple LR method.

1.1 What Does "Shift-Reduce" Mean?

A shift-reduce parser keeps a stack of symbols (or, in LR parsing, states) and repeatedly does one of two things:

1. **Shift** — take the next input symbol and push it onto the stack.
2. **Reduce** — recognize that the top of the stack matches the right-hand side (body) of some production, pop that body off, and push the left-hand side (head) of the production instead.

The parser keeps doing this until either (a) the stack contains just the start symbol and the input is empty — success (accept) — or (b) no shift and no reduce is legal — a syntax error.

1.2 From Grammar Symbols to States

A plain shift-reduce parser shifts grammar **symbols** onto its stack. An LR parser is smarter: instead of shifting the raw symbol, it shifts a **state** that summarizes everything the parser needs to remember about the stack contents below and including that symbol. Each state corresponds to a set of "items" (partially-matched productions) from an automaton that was built from the grammar ahead of time — this automaton is exactly the LR(0) automaton built earlier for the grammar.

Tip: Whenever a question says "draw/construct the LR(0) automaton first," it is asking you to build this same table of item-sets and GOTO transitions that everything in SLR parsing is based on.

1.3 The Model of an LR Parser

An LR parser has five parts:

- **Input** — the buffer holding the remaining tokens of the string, terminated by the end-marker \$.
- **Output** — the sequence of productions used in the reduction (this is what lets us build the parse tree / derivation).

- **Stack** — holds a sequence of states $s_0 s_1 \dots s_m$, where s_m is on top. s_0 is always the special initial state and is never popped; it acts like a "floor" under the stack.
- **Driver program** — the same fixed control loop for every grammar; only the table changes from grammar to grammar.
- **Parsing table** — has two parts, ACTION and GOTO (explained below). This is the part we must construct differently for every grammar.

Important fact: because states are built from sets of items, and because there is a transition from state i to state j exactly when $\text{GOTO}(I_i, X) = I_j$ for some grammar symbol X , every state (except the start state 0) has a unique grammar symbol linked to it — the symbol that was "consumed" to reach that state. Two different states may share the same symbol (for example, in the classic $\text{id} * \text{id} + \text{id}$ grammar, both state 1 and state 8 are entered by a transition on E), but that does not break anything.

1.4 Review Questions

Q: Define an LR parser.

A: A bottom-up parser that scans input Left-to-right and builds a Rightmost derivation in reverse, using a stack of states, a driver program, and an ACTION/GOTO table to decide between shifting, reducing, accepting, or erroring at each step.

Q: What are the advantages of LR parsing in general?

A: Detects errors as early as possible; can be built for a very large class of unambiguous grammars (larger than LL); does not require backtracking; naturally supports left-recursive grammars, which LL parsers cannot handle directly.

Q: What is an application of LR parsing?

A: It is the parsing technique used inside most real compiler-generator tools (e.g. the LALR method is used by classic tools such as yacc/bison) for parsing full programming-language grammars.

2. Structure of the LR Parsing Table

The parsing table has exactly two parts:

2.1 The ACTION Function

ACTION takes two arguments: a state i (from the top of the stack) and a terminal a (or the end-marker $\$$). $\text{ACTION}[i, a]$ can be exactly one of four things:

3. **Shift j** — shift the next input symbol onto the stack, but represent it using state j (not the raw symbol).
4. **Reduce $A \rightarrow \beta$** — the top of the stack matches β (the body of a production); pop $|\beta|$ states, expose the state below, and push the GOTO state for the head A .
5. **Accept** — parsing finished successfully.
6. **Error** — the blank/empty entries. The parser has found a syntax error and must call an error-recovery routine (see Section 6 of this note).

2.2 The GOTO Function

GOTO was originally defined on sets of items ($\text{GOTO}(I, X) = I_j$). We simply relabel it in terms of state numbers: if $\text{GOTO}(I, A) = I_j$ for a nonterminal A , then $\text{GOTO}[i, A] = j$. This part of the table is only ever consulted for nonterminals — it tells the parser which state to push right after a reduction, once the head of the production has been recognised.

Tip: A very common exam mistake is looking up a terminal in the GOTO columns, or a nonterminal in the ACTION columns. Terminals (and \$) always live under ACTION; nonterminals always live under GOTO. The two parts of the table answer two different questions: "what do I do with the next input symbol" versus "where do I go after I just built this nonterminal."

2.3 LR-Parser Configurations

To talk precisely about "where the parser currently is," we use a **configuration** — a pair $(s_0 s_1 \dots s_m, a_1 a_{i+1} \dots a_n \$)$, where the first component is the current stack contents (top written on the right) and the second component is the remaining, unread input.

This configuration represents the right-sentential form $X_1 X_2 \dots X_m a_i a_{i+1} \dots a_n$, exactly the way a plain shift-reduce parser would represent it — except that instead of storing the grammar symbols $X_1 \dots X_m$ directly, the stack stores the states from which those symbols can be recovered (since each state — other than state 0 — is tied to one particular grammar symbol, as explained in Section 1.3).

- s_0 , the start state, is a bottom-of-stack marker. It does not correspond to any grammar symbol, but it plays an important role because GOTO/ACTION lookups always ultimately rest on it.

2.4 Review Questions

Q: Define the ACTION function.

A: A function taking a state i and a terminal (or $\$$) a , returning one of: shift j , reduce $A \rightarrow \beta$, accept, or error.

Q: Define the GOTO function (as used in the parsing table).

A: A function taking a state i and a nonterminal A , returning the state j to push right after a reduction to A , where $\text{GOTO}(I_i, A) = I_j$ in the underlying LR(0) automaton.

Q: Define a configuration of an LR parser.

A: A pair $(s_0 s_1 \dots s_m, a_i a_{i+1} \dots a_n \$)$ consisting of the current stack contents (states, top on the right) and the remaining unread input.

Q: What are the four possible values of ACTION[i,a]?

A: Shift j , Reduce $A \rightarrow \beta$, Accept, or Error.

Q: What table is consulted to detect an error?

A: Only the ACTION table. Errors are never detected by consulting GOTO.

Q: Differentiate ACTION and GOTO.

A: ACTION is indexed by (state, terminal-or- $\$$) and used on every parser step to decide the next move (shift/reduce/accept/error). GOTO is indexed by (state, nonterminal) and is consulted only immediately after a reduce, to decide which state to push for the newly-formed nonterminal.

Q: Why not look up a terminal symbol in the GOTO half of the table?

A: Because GOTO is defined only over nonterminals — $\text{GOTO}(li, A)$ is meaningful only when A is a nonterminal that has just been produced by a reduction; terminal lookups always belong in ACTION.

3. Behavior of the LR Parser — the Four Moves

Given a configuration as above, the parser looks at the current input symbol a_i and the state sm on top of the stack, then consults $\text{ACTION}[sm, a_i]$. Exactly one of four things happens.

3.1 Shift Move

If $\text{ACTION}[sm, a_i] = \text{shift } s$, the parser:

7. Pushes the new state s onto the stack.
8. Advances the input pointer to a_{i+1} (the next input symbol).

New configuration: $(s_0 s_1 \cdots sm\ s, a_{i+1} \cdots a_n \$)$.

Note that the actual symbol a_i itself is never stored separately on the stack — it can always be recovered later from the state s if needed (in practice it is never needed, because every state already "remembers" everything relevant).

3.2 Reduce Move

If $\text{ACTION}[sm, a_i] = \text{reduce } A \rightarrow \beta$, and $r = |\beta|$ (the length of the production body), the parser:

9. Pops r states off the stack, exposing state $sm-r$.
10. Looks up $s = \text{GOTO}[sm-r, A]$ and pushes s onto the stack.
11. Outputs (prints / records) the production $A \rightarrow \beta$ as one more step of the reduction sequence.

New configuration: $(s_0 s_1 \cdots sm-r\ s, a_i a_{i+1} \cdots a_n \$)$.

Notice: the current input symbol a_i is NOT consumed by a reduce move. Only a shift move consumes input. This is one of the most important facts to remember for simulation questions — after a reduce, you re-examine the same input symbol against the new state.

Tip: The sequence of grammar symbols $X_{m-r+1} \dots X_m$ that correspond to the r popped states will always exactly match β , the right-hand side of the reducing production — this is guaranteed by the correctness of the automaton, so if what you popped does not look like the production body, you have made an arithmetic or table-lookup mistake somewhere earlier.

3.3 Accept

If $\text{ACTION}[sm, a_i] = \text{accept}$, parsing is complete and the string is grammatically valid. This only happens when the stack holds state 0 followed by the state for the (augmented) start symbol, and the remaining input is just $\$$.

3.4 Error

If $\text{ACTION}[sm, a_i]$ is blank/undefined, the parser has discovered a syntax error and calls an error-recovery routine (panic-mode or phrase-level — discussed later in Section 6).

3.5 The Driver-Program Pseudocode

The full LR-parsing algorithm, which is identical for every grammar (only the table differs), is:

```

let a be the first symbol of w$; while (1) {                                     /*
repeat forever */    let s be the state on top of the stack;    if
(ACTION[s, a] == shift t) {    push t onto the stack;    let a be
the next input symbol;    } else if (ACTION[s, a] == reduce A -> beta) {
pop |beta| symbols off the stack;    let state t now be on top of the
stack;    push GOTO[t, A] onto the stack;    output the production
A -> beta;    } else if (ACTION[s, a] == accept) break;    /* parsing done */
else call error-recovery routine; }

```

Input: a string w and an LR-parsing table with functions ACTION and GOTO for a grammar G .

Output: if w is in $L(G)$, the sequence of reduction steps of a bottom-up parse for w ; otherwise, an error indication.

Method: initially the parser has s_0 (the initial state) on its stack, and $w\$$ in the input buffer. It then repeatedly executes the loop shown above.

3.6 Review Questions

Q: What does the stack of an LR parser actually contain?

A: A sequence of states from the LR(0) automaton (not grammar symbols directly) — s_0 at the bottom (never popped) up to sm on top.

Q: What happens to the current input symbol during a reduce move?

A: Nothing — it is not consumed. The same symbol is examined again immediately afterward, now against the newly-exposed top-of-stack state.

Q: What is popped during a reduce $A \rightarrow \beta$?

A: Exactly $|\beta|$ states are popped off the stack (and correspondingly $|\beta|$ grammar symbols, if you are also tracking a Symbol column).

Q: What is pushed after a reduce?

A: The state $\text{GOTO}[t, A]$, where t is the state exposed on the stack immediately after popping, and A is the head of the reducing production.

Q: How many states does a production's reduce action pop from the stack?

A: Exactly the number of grammar symbols on the right-hand side of that production ($|\beta|$); an ϵ -production pops zero states.

Q: How does an LR parser decide between shift and reduce at each step?

A: It consults ACTION[top-of-stack state, current input symbol]; whichever single action is stored there (shift, reduce, accept, or error) is executed — there is no separate decision process beyond this table lookup.

Q: Compare shift and reduce moves.

A: Shift consumes one input symbol and pushes a new state derived from it; it does not touch any earlier stack contents. Reduce consumes zero input symbols but pops $|\beta|$ existing states/symbols from the stack and replaces them with a single new state derived from GOTO on the production's head.

Q: Why not just accept as soon as the input is empty, without checking the stack contents?

A: Because the input could be empty while the stack still holds an incomplete or malformed sequence of states — for example, halfway through matching if(Cond) with the Block missing. Only a specific, well-defined stack shape (state 0 followed by the accept-triggering state) combined with empty remaining input guarantees a fully successful parse.

Q: Explain the complete life-cycle of a single reduce move, from table lookup to the next iteration of the parsing loop.

A: (1) The driver examines the state s on top of the stack and the current input symbol a , and finds ACTION[s, a] = reduce $A \rightarrow \beta$. (2) It computes $r = |\beta|$. (3) It pops r states off the stack, which by construction correspond exactly to the grammar symbols of β , exposing a state t underneath. (4) It looks up GOTO[t, A] = s' and pushes s' onto the stack. (5) It records/outputs the production $A \rightarrow \beta$ as the next step of the reduction sequence (this is exactly what eventually lets us reconstruct the full parse tree or the rightmost derivation in reverse). (6) Critically, the current input symbol a is NOT advanced — the very next iteration of the driver loop re-examines the same a , now against the new top-of-stack state s' , to decide the next move. This is what allows chains of several reduces to happen back-to-back on a single input symbol before a shift finally consumes it.

4. Worked Example — LR Parsing Table for the Expression Grammar

We use this grammar throughout this note as the running example.

4.1 The Grammar (numbered)

#	Production
(1)	$E \rightarrow E + T$
(2)	$E \rightarrow T$
(3)	$T \rightarrow T * F$
(4)	$T \rightarrow F$
(5)	$F \rightarrow (E)$
(6)	$F \rightarrow \text{id}$

The numbers matter because reduce actions in the table are written as "rj" meaning "reduce by production number j", not by writing out the whole production.

4.2 The ACTION / GOTO Table

Codes used in the table:

- si — shift, and stack (push) state i.
- rj — reduce by the production numbered j.
- acc — accept.
- blank — error.

State	id	+	*	(	)	\$	E	T	F
0	s5			s4			1	2	3
1		s6				acc			
2		r2	s7		r2	r2			
3		r4	r4		r4	r4			
4	s5			s4			8	2	3
5		r6	r6		r6	r6			
6	s5			s4				9	3
7	s5			s4					10
8		s6			s11				
9		r1	s7		r1	r1			
10		r3	r3		r3	r3			
11		r5	r5		r5	r5			

4.3 Reading a Cell — a Beginner's Walkthrough

Take ACTION[0, id] = s5. This is read as: "if the parser is in state 0 and the next input token is id, shift and go to state 5." Take GOTO[0, E] = 1 (found under the E column, in the GOTO half of the same table): "once we reduce something to E while state 0 is exposed underneath, push state 1." Every single cell in this table was produced mechanically from the LR(0) automaton plus the FOLLOW sets, using the algorithm in Section 5 below — nothing in the table is guessed or memorised; it is all derived.

4.4 Simulating id * id + id — A Full Worked Trace

The table below shows the full sequence of stack/symbol/input/action lines for parsing the string id * id + id. Study this table line by line; it is one of the most frequently asked simulation questions.

Line	Stack	Symbol	Input	Action
(1)	0	\$	id * id + id \$	shift

Line	Stack	Symbol	Input	Action
(2)	0 5	\$ id	* id + id \$	reduce by $F \rightarrow id$
(3)	0 3	\$ F	* id + id \$	reduce by $T \rightarrow F$
(4)	0 2	\$ T	* id + id \$	shift
(5)	0 2 7	\$ T *	id + id \$	shift
(6)	0 2 7 5	\$ T * id	+ id \$	reduce by $F \rightarrow id$
(7)	0 2 7 10	\$ T * F	+ id \$	reduce by $T \rightarrow T * F$
(8)	0 2	\$ T	+ id \$	reduce by $E \rightarrow T$
(9)	0 1	\$ E	+ id \$	shift
(10)	0 1 6	\$ E +	id \$	shift
(11)	0 1 6 5	\$ E + id	\$	reduce by $F \rightarrow id$
(12)	0 1 6 3	\$ E + F	\$	reduce by $T \rightarrow F$
(13)	0 1 6 9	\$ E + T	\$	reduce by $E \rightarrow E + T$
(14)	0 1	\$ E	\$	accept

Line-by-line reasoning

- Line (1): stack has just state 0, current symbol is id. $ACTION[0, id] = s5$, so we shift and push 5. Nothing is reduced yet because we have not seen a complete production body.
- Line (2): top state is 5, current symbol is *. $ACTION[5, *] = r6$, i.e. reduce by $F \rightarrow id$. We pop 1 state (id has length 1), exposing state 0, then $GOTO[0, F] = 3$, so we push 3.
- Line (3): top state is 3, symbol is *. $ACTION[3, *] = r4$, reduce by $T \rightarrow F$. Pop 1 state, expose 0, $GOTO[0, T] = 2$, push 2.
- Line (4): top state 2, symbol *. $ACTION[2, *] = s7$ (this time it is a shift, not a reduce, because $T \rightarrow T * F$ is still "open" — the dot is right before *). Shift, push 7.
- Line (5): top state 7, symbol id. $ACTION[7, id] = s5$, shift, push 5.
- Line (6): top state 5, symbol +. $ACTION[5, +] = r6$, reduce by $F \rightarrow id$. Pop 1, expose 7, $GOTO[7, F] = 10$, push 10.
- Line (7): top state 10, symbol +. $ACTION[10, +] = r3$, reduce by $T \rightarrow T * F$. The body $T * F$ has 3 symbols, so we pop exactly 3 states off the stack "0 2 7 10" — namely 10, 7, and 2 (representing F, *, and T). This exposes state 0. We then look up $GOTO[0, T] = 2$ and push 2, giving stack "0 2".
- Line (8): top state 2, symbol +. $ACTION[2, +] = r2$, reduce by $E \rightarrow T$. Pop 1, expose 0, $GOTO[0, E] = 1$, push 1.
- Line (9): top state 1, symbol +. $ACTION[1, +] = s6$, shift, push 6.
- Line (10): top state 6, symbol id. $ACTION[6, id] = s5$, shift, push 5.
- Line (11): top state 5, symbol \$. $ACTION[5, $] = r6$, reduce by $F \rightarrow id$. Pop 1, expose 6, $GOTO[6, F] = 3$, push 3.

- Line (12): top state 3, symbol \$. ACTION[3, \$] = r4, reduce by $T \rightarrow F$. Pop 1, expose 6, GOTO[6, T] = 9, push 9.
- Line (13): top state 9, symbol \$. ACTION[9, \$] = r1, reduce by $E \rightarrow E + T$. Pop 3 (E, +, T), expose 0, GOTO[0, E] = 1, push 1.
- Line (14): top state 1, symbol \$. ACTION[1, \$] = acc. Done — the string is accepted.

Tip: Whenever you pop for a reduce, count exactly $|\beta|$ states/symbols — not "however many look convenient". A very common exam mistake is popping the wrong number of states for productions with 3 symbols on the right (like $T \rightarrow T * F$ or $E \rightarrow E + T$), which then corrupts every subsequent GOTO lookup.

5. Constructing SLR(1) Parsing Tables

"SLR" stands for **Simple LR**. The SLR method is the simplest of the three main LR table-construction methods (SLR, Canonical LR, LALR). It starts from the plain LR(0) automaton — the same automaton you already know how to build — and adds just one extra ingredient: FOLLOW sets, used only to decide when a reduce action is legal.

5.1 Step 0 — Augment the Grammar

Given the original grammar G with start symbol S , we build the **augmented grammar** G' by adding one brand-new production $S' \rightarrow S$, where S' is a fresh symbol that does not occur in G . This single extra production tells the parser unambiguously when to accept: acceptance happens exactly when the item $S' \rightarrow S$ is reached with lookahead \$, i.e. the entire original start symbol has been reduced and nothing else remains.

Tip: Without augmentation, the parser could sometimes reduce S and then not know whether to stop (accept) or keep going, especially if S itself could appear inside a larger sentential form. Augmentation removes that ambiguity.

5.2 Step 1 — Build the Canonical Collection of LR(0) Items

Construct $C = \{I_0, I_1, \dots, I_n\}$, the canonical collection of sets of LR(0) items for G' , together with the GOTO function between them: start with $\text{CLOSURE}(\{[S' \rightarrow \cdot S]\})$ as I_0 , then repeatedly apply GOTO on every grammar symbol to discover new item sets, until no more new sets appear.

5.3 Step 2 — Fill in the ACTION Table

State i is built from item set I_i . For every item in I_i , do the following:

12. (a) If $[A \rightarrow \alpha \cdot a\beta]$ is in I_i , and $\text{GOTO}(I_i, a) = I_j$, where a is a TERMINAL, then set $\text{ACTION}[i, a] = \text{"shift } j\text{"}$.
13. (b) If $[A \rightarrow \alpha \cdot]$ is in I_i (a completed item, i.e. the dot is at the very end), then set $\text{ACTION}[i, a] = \text{"reduce } A \rightarrow \alpha\text{"}$ for every terminal a in $\text{FOLLOW}(A)$. (Here A must not be S' — the augmented start symbol is handled separately by rule (c) below.)
14. (c) If $[S' \rightarrow S \cdot]$ is in I_i , then set $\text{ACTION}[i, \$] = \text{"accept\text{"}$.

If any of these rules ever try to write two different actions into the same cell (for example both a shift and a reduce, or two different reduces), the grammar is NOT SLR(1) and the algorithm fails to build a

valid parser for that cell — this situation is called a shift/reduce conflict or a reduce/reduce conflict, discussed in Section 5.6.

5.4 Step 3 — Fill in the GOTO Table

For every nonterminal A , if $\text{GOTO}(li, A) = lj$, set $\text{GOTO}[i, A] = j$. (Recall this is exactly the same GOTO function as in the LR(0) automaton — we are simply relabelling item sets by their state numbers.)

5.5 Step 4 & 5 — Errors and the Start State

- Step 4: every entry not defined by rules (2) or (3) above is left as "error" (a blank cell).
- Step 5: the initial state of the parser is the state built from the item set that contains $[S' \rightarrow \cdot S]$ — i.e. state 0, by convention.

The table built by this whole procedure is called the SLR(1) table for G , and an LR parser using it is called the SLR(1) parser for G . A grammar with a valid (conflict-free) SLR(1) table is said to be an SLR(1) grammar. We usually drop the "(1)" and just say "SLR" because we will not study parsers with more than 1 symbol of lookahead in this course.

5.6 Building the Table Cell by Cell — A Worked Example

Let us build a few entries of the ACTION/GOTO table directly from the item sets, to see the algorithm in action.

Item set I0

$I_0 = \text{CLOSURE}(\{E' \rightarrow \cdot E\})$ which expands (by repeatedly adding productions for whichever nonterminal follows a dot) to:

- $E' \rightarrow \cdot E$
- $E \rightarrow \cdot E + T$
- $E \rightarrow \cdot T$
- $T \rightarrow \cdot T * F$
- $T \rightarrow \cdot F$
- $F \rightarrow \cdot (E)$
- $F \rightarrow \cdot \text{id}$

The item $F \rightarrow \cdot (E)$ has a terminal, (, right after the dot, and $\text{GOTO}(I_0, ()$ leads to some state (state 4 in our numbering). This gives $\text{ACTION}[0, (] = \text{shift } 4$. Likewise $F \rightarrow \cdot \text{id}$ gives $\text{ACTION}[0, \text{id}] = \text{shift } 5$. None of the other items in I_0 have a terminal directly after the dot (E, T, F are all nonterminals), so they contribute nothing to ACTION — but they do contribute to GOTO: $\text{GOTO}(I_0, E)$ leads to state 1, $\text{GOTO}(I_0, T)$ leads to state 2, $\text{GOTO}(I_0, F)$ leads to state 3, giving $\text{GOTO}[0, E]=1$, $\text{GOTO}[0, T]=2$, $\text{GOTO}[0, F]=3$, exactly matching row 0 of the table.

Item set I1

$I_1 = \text{GOTO}(I_0, E) = \{E' \rightarrow E \cdot, E \rightarrow E \cdot + T\}$.

The first item is complete for E' , so by rule (c) $\text{ACTION}[1, \$] = \text{accept}$. The second item has a terminal (+) after the dot, and $\text{GOTO}(11, +)$ leads to state 6, giving $\text{ACTION}[1, +] = \text{shift 6}$.

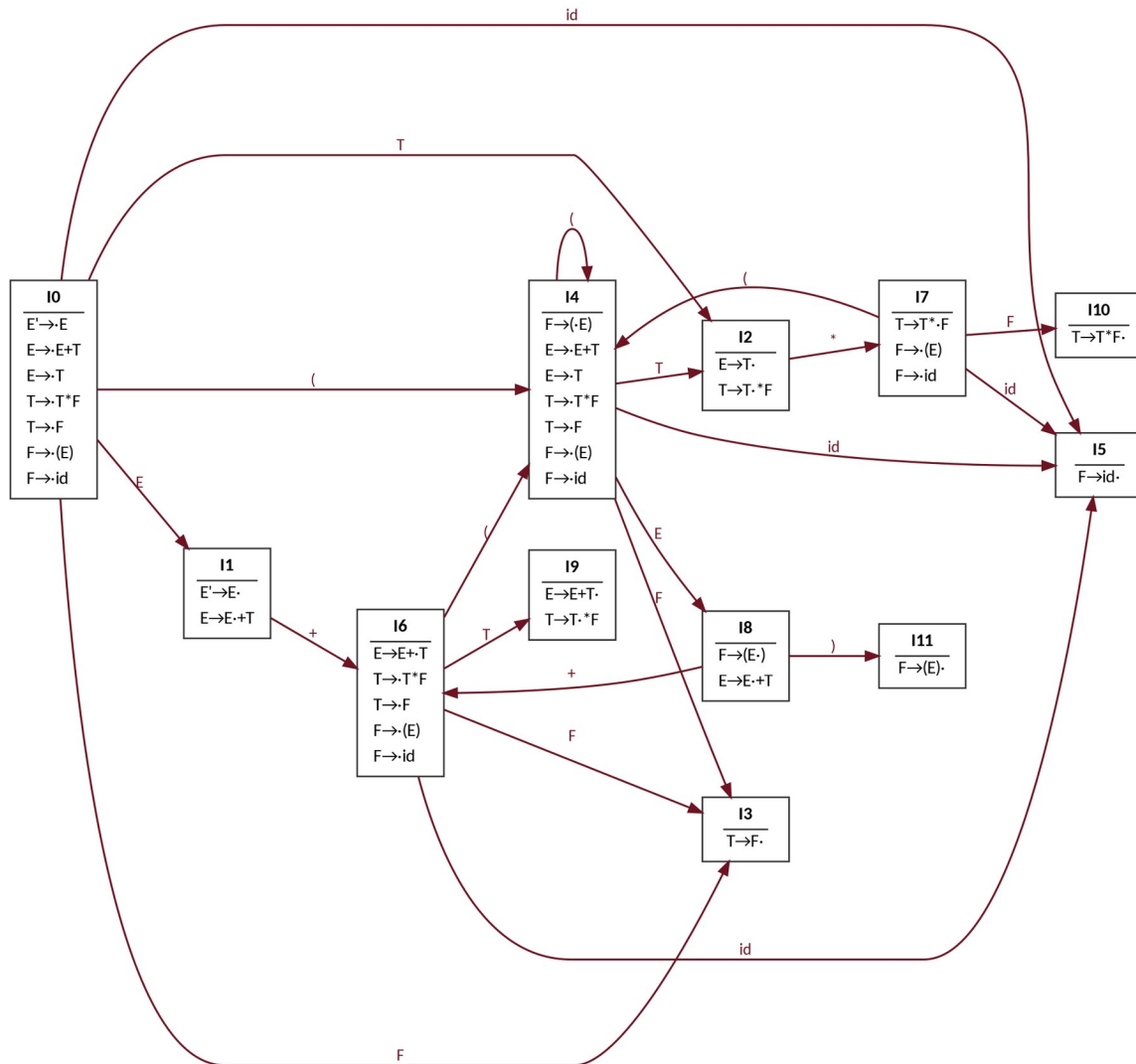
Item set I2

$I2 = \text{GOTO}(I0, T) = \{E \rightarrow T \cdot, T \rightarrow T \cdot * F\}$.

The first item, $E \rightarrow T \cdot$, is a completed item for nonterminal E , so by rule (b) we add "reduce $E \rightarrow T$ " to $\text{ACTION}[2, a]$ for every $a \in \text{FOLLOW}(E)$. Since $\text{FOLLOW}(E) = \{\$, +,)\}$, we get $\text{ACTION}[2, \$] = \text{ACTION}[2, +] = \text{ACTION}[2,)] = \text{reduce } E \rightarrow T$ (production 2). The second item has $*$ right after the dot, and $\text{GOTO}(I2, *)$ leads to state 7, giving $\text{ACTION}[2, *] = \text{shift 7}$.

Continuing this exact process for every item set gives every cell of the full ACTION/GOTO table shown earlier — nothing is arbitrary.

Complete LR(0) Automaton for the Running Example (All 12 States)



5.7 Why FOLLOW Sets Matter So Much Here

Rule (b) is the **only** place where FOLLOW sets are used in this whole algorithm. A completed item $[A \rightarrow \alpha \cdot]$ tells us "we could reduce right now" — but reducing is only safe on those lookahead symbols that could genuinely follow A somewhere in a valid sentential form. If we allowed a reduce on every possible terminal, we would sometimes reduce when we should have kept shifting, producing wrong or blocked parses. FOLLOW(A) exactly characterises the safe lookahead symbols for reducing to A (under the SLR approximation — the Canonical-LR and LALR methods use a more precise, item-specific lookahead instead of the grammar-wide FOLLOW set, which is why they can parse a strictly larger class of grammars, as shown in the next example below).

5.8 A Grammar That Is Unambiguous but NOT SLR(1)

This is one of the most important "why not" examples in SLR parsing, because it shows that SLR(1) is strictly weaker than the other LR methods.

Grammar:

- $S \rightarrow L = R \mid R$
- $L \rightarrow * R \mid \text{id}$
- $R \rightarrow L$

(Think of L as an l-value — something that can appear on the left of an assignment — and R as an r-value — a value that can be stored somewhere. * denotes "contents of", i.e. dereference.)

The conflict

Consider item set $I_2 = \{S \rightarrow L \cdot = R, R \rightarrow L \cdot\}$. The first item has = right after the dot, a terminal, so it contributes ACTION[2, =] = shift 6. The second item, $R \rightarrow L \cdot$, is a completed item for R, so rule (b) adds "reduce $R \rightarrow L$ " for every a in FOLLOW(R). Now, FOLLOW(R) does contain = — to see why, consider the derivation $S \Rightarrow L = R \Rightarrow *R = R$: here R is immediately followed by =. So rule (b) also wants ACTION[2, =] = reduce $R \rightarrow L$.

We now have BOTH a shift and a reduce demanded for the same cell ACTION[2, =]. This is called a **shift/reduce conflict**. The SLR algorithm cannot resolve it, so it declares that this grammar is not SLR(1).

Why isn't the grammar itself ambiguous, then?

This grammar is perfectly unambiguous — every string it generates has exactly one parse tree. The conflict is not a flaw in the grammar; it is a limitation of the SLR method itself. The SLR construction is "not powerful enough to remember enough left context" to decide correctly on input = after having seen a string reducible to L. In plain English: by the time the parser is in state 2, it has forgotten whether the L it is holding came from the very beginning of the sentential form (in which case = should be shifted, because we are looking at $S \rightarrow L \cdot = R$) or from somewhere where reducing $R \rightarrow L$ is actually needed. Both the Canonical-LR and the LALR methods carry extra, item-specific lookahead information (instead of using one grammar-wide FOLLOW(R) for every occurrence of a completed R-item) and can correctly resolve this case, so they succeed on a strictly larger collection of grammars than SLR.

Tip: There is also a general fact worth remembering for viva/short-answer questions: every SLR(1) grammar is unambiguous, but the converse is false — there exist unambiguous grammars that are not

SLR(1) (this example), and there even exist unambiguous grammars for which every LR construction method (SLR, LALR, Canonical LR) produces conflicts. Fortunately such worst-case grammars rarely show up in real programming-language design.

5.9 Review Questions

Q: Define SLR(1).

A: "Simple LR with 1 symbol of lookahead" — an LR table-construction method that builds ACTION/GOTO directly from the plain LR(0) automaton, using the grammar-wide FOLLOW sets (rather than item-specific lookaheads) to decide when reduces are legal.

Q: Define an augmented grammar.

A: The original grammar G with one new production $S' \rightarrow S$ added, where S' is a brand-new start symbol not used anywhere else, so that acceptance can be detected unambiguously.

Q: What ingredient does SLR add on top of a plain LR(0) automaton?

A: FOLLOW sets — used only to decide, for each completed item, on which lookahead symbols a reduce is actually legal.

Q: What is the only rule in SLR table construction that uses FOLLOW sets?

A: Rule (b): for a completed item $[A \rightarrow \alpha \cdot]$ in state i , set $\text{ACTION}[i, a] = \text{reduce } A \rightarrow \alpha$ for every terminal a in $\text{FOLLOW}(A)$.

Q: How is FOLLOW(A) used differently from FIRST(A) in SLR table construction?

A: FIRST is not used directly in the SLR algorithm at all (it is used earlier, to build LL parsers and, indirectly, in computing FOLLOW sets); FOLLOW(A) is used exclusively in rule (b), to restrict which lookahead symbols permit a reduce to A .

Q: State the rule for when ACTION[i, a] gets a shift entry.

A: Whenever an item $[A \rightarrow \alpha \cdot a \beta]$ is in li and $\text{GOTO}(li, a) = lj$ for a terminal a , set $\text{ACTION}[i, a] = \text{shift } j$.

Q: State the rule for when ACTION[i, a] gets a reduce entry.

A: Whenever an item $[A \rightarrow \alpha \cdot]$ (a completed item, $A \neq S'$) is in li , set $\text{ACTION}[i, a] = \text{reduce } A \rightarrow \alpha$ for every terminal a in $\text{FOLLOW}(A)$.

Q: State the exception case for the SLR reduce-action rule (rule (b)).

A: The rule explicitly excludes $A = S'$ (the augmented start symbol); its completed item $[S' \rightarrow S \cdot]$ is instead handled only by rule (c), which sets $\text{ACTION}[i, \$] = \text{accept}$.

Q: What is a shift/reduce conflict?

A: A single ACTION table cell that the construction algorithm tries to fill with both a shift action and a reduce action simultaneously.

Q: What is a reduce/reduce conflict?

A: A single ACTION table cell that the construction algorithm tries to fill with two different reduce actions (from two different completed items whose FOLLOW sets overlap on that symbol).

Q: Differentiate a shift/reduce conflict from a reduce/reduce conflict.

A: A shift/reduce conflict occurs when the same table cell wants both a shift action and a reduce action. A reduce/reduce conflict occurs when the same cell wants two different reduce actions (from two distinct completed items whose FOLLOW sets both contain that symbol).

Q: What condition on the ACTION table makes a grammar fail to be SLR(1)?

A: Any cell of the table being assigned two or more conflicting actions by the construction rules (a shift/reduce or reduce/reduce conflict).

Q: What must be true for the grammar to be called SLR(1)?

A: Every cell of the ACTION/GOTO table produced by the SLR construction must be uniquely defined — no conflicts anywhere.

Q: When is a grammar considered SLR(1)?

A: When the SLR construction, applied to the augmented grammar, produces a parsing table with no conflicting cells at all.

Q: Why is the grammar augmented with $S' \rightarrow S$ before building the table?

A: So that the parser has one single, unambiguous item, $[S' \rightarrow S \cdot]$, whose presence with lookahead \$ can be used as the sole trigger for the accept action — without augmentation the parser could not always tell whether reducing S meant "done" or "keep going."

Q: Why is the lookahead \$ specifically required for $[S' \rightarrow S \cdot]$ to trigger acceptance, rather than accepting as soon as the stack just holds the start symbol?

A: Because S itself might still be a valid prefix of a longer sentential form in some grammars if it could be embedded inside other constructs; requiring lookahead \$ guarantees that genuinely nothing is left in the input, which is the only situation in which stopping is actually correct. Since S' is brand new and used nowhere else, $[S' \rightarrow S \cdot]$ only ever appears when the ENTIRE input has legitimately reduced to the original start symbol, and \$ confirms no more input remains.

Q: Why does a production like $A \rightarrow \epsilon$ get a completed item immediately, with the dot never having moved?

A: Because the right-hand side is empty; the item $A \rightarrow \cdot$ represents "the body has zero symbols and all zero of them have already been matched," so it is complete the instant it is added to an item set by CLOSURE — there is no shifting needed to reach the end of an empty body.

Q: Why does an ϵ -production often force extra reduce entries across many table rows?

A: Because CLOSURE adds the completed item $A \rightarrow \cdot$ to every state where A could start (wherever a production has A right after the dot), and the SLR reduce-action rule then adds a reduce action for $A \rightarrow \epsilon$ to that state for every symbol in $\text{FOLLOW}(A)$ — since $\text{FOLLOW}(A)$ is often a wide set (e.g., {if, while, a, \$} in Problem A), this shows up as several reduce entries scattered through many different states.

Q: Why does SLR sometimes fail on grammars that are not actually ambiguous?

A: Because SLR uses one grammar-wide $\text{FOLLOW}(A)$ for every occurrence of a completed A -item, regardless of context — it cannot remember which specific earlier construct led to that state. When two different contexts both lead to states with a completed A -item, but only one of those contexts should legally reduce on some lookahead symbol, SLR cannot tell them apart and reports a conflict, even though a smarter, context-sensitive lookahead (as used by canonical-LR/LALR) could resolve it correctly.

Q: When should you use the LALR/Canonical-LR method instead of SLR?

A: Whenever the SLR construction produces conflicts on a grammar you believe is genuinely unambiguous — this is a strong signal that the grammar needs the extra, item-specific lookahead precision that LALR/Canonical-LR provide.

Q: What are the advantages of SLR specifically, compared to canonical LR?

A: The parsing table is much smaller (built directly from the plain LR(0) automaton, with no need to track separate lookahead sets per item), and construction is considerably simpler and faster.

Q: What is a disadvantage/limitation of SLR?

A: It can report spurious conflicts on grammars that are genuinely unambiguous, simply because it approximates reduce-lookahead with a single grammar-wide FOLLOW set instead of more precise, context-specific lookahead.

Q: What is a limitation shared by every LR parser-construction method?

A: There exist unambiguous grammars for which even the most powerful LR method (canonical LR) still produces parsing-action conflicts — no LR method can handle every unambiguous grammar, although such worst-case grammars are rare in real programming-language design.

Q: State a key property linking SLR(1) grammars and unambiguous grammars.

A: Every SLR(1) grammar is unambiguous, but the converse is false: some unambiguous grammars are not SLR(1).

Q: Why can SLR/LALR parsers make several reductions before announcing a syntax error, while canonical-LR parsers cannot make even one?

A: Because SLR and LALR merge lookahead information across states in ways that are sometimes less precise than the state actually warrants, so an invalid symbol may still trigger one or more "legal-looking" reduces before the true mismatch is finally exposed at the ACTION table. A canonical-LR parser keeps fully precise, item-specific lookahead, so it detects the very first invalid symbol immediately, without any intervening reduces.

Q: Compare LR(0), SLR(1), and canonical LR(1)/LALR(1) in one line each.

A: LR(0) uses no lookahead at all for reduce decisions (very weak, rarely used directly). SLR(1) reuses the LR(0) automaton but adds grammar-wide FOLLOW sets as lookahead. Canonical LR(1)/LALR(1) carry item-specific lookahead sets computed alongside the automaton itself, so they can resolve conflicts that SLR cannot; LALR(1) is a space-saving compromise that merges states with identical cores while (mostly) keeping canonical LR's precision.

Q: Explain, with reference to viable prefixes and valid items, why FOLLOW-set-based lookahead is only an approximation, and describe a concrete grammar where this approximation fails.

A: FOLLOW(A) is computed purely from the grammar's productions, independent of which specific parser state or context we are in — it lumps together every possible situation in which A could ever be followed by a given terminal, anywhere in the grammar. But the actual, precise question the parser needs answered at a completed item $[A \rightarrow \alpha \cdot]$ is narrower: "given exactly how we got to this specific state (this specific viable prefix), which terminals can legally follow this particular occurrence of A?" These two questions coincide only when every context that can reach a completed A-item happens to agree on which terminals may follow it. Consider the grammar $S \rightarrow L = R \mid R, L \rightarrow *R \mid id, R \rightarrow L$ as a counter-example:

state $I_2 = \{S \rightarrow L \cdot R, R \rightarrow L \cdot\}$ is reached whenever an L has just been recognised, but that L might be the target of an assignment (in which case $\cdot$ should be shifted) or might already be a complete r -value R (in which case $R \rightarrow L$ should be reduced even though the very next symbol is $\cdot$, since $\text{FOLLOW}(R)$ also contains $\cdot$). SLR merges both situations into one grammar-wide $\text{FOLLOW}(R)$ and cannot separate them, producing an unavoidable shift/reduce conflict, even though the grammar itself is perfectly unambiguous.

6. Viable Prefixes — Why the LR(0) Automaton Is Trustworthy

This section answers the deeper "why does this even work" question that examiners like to ask.

The LR(0) automaton for a grammar characterises exactly the strings of grammar symbols that are allowed to sit on the stack of a shift-reduce parser at any point during a correct parse. Formally, if the stack holds α and the remaining input is x , then a sequence of reductions must be able to take αx all the way to S (the start symbol) — written $S \Rightarrow^*_{rm} \alpha x$.

Not every prefix of a right-sentential form is allowed to sit on the stack, though — the parser must never shift past the **handle** (the substring that should be reduced next). For example, if $F \rightarrow (E)$, then at various points the stack may legally hold $($, then $(E$, then (E) , — but it must never hold $(E)^*$, because (E) is itself a handle that has to be reduced to F before the $*$ can be shifted.

The prefixes that ARE allowed on the stack are called **viable prefixes**: a viable prefix is a prefix of a right-sentential form that does not extend past the right end of that sentential form's rightmost handle. Equivalently, it is always possible to append more terminal symbols to a viable prefix and still get a valid right-sentential form.

We say an item $A \rightarrow \beta_1 \cdot \beta_2$ is **valid** for a viable prefix $\alpha\beta_1$ if there is a rightmost derivation $S' \Rightarrow^*_{rm} \alpha A w \Rightarrow_{rm} \alpha\beta_1\beta_2 w$. In general, one item can be valid for many different viable prefixes, and one viable prefix can have many valid items (this is exactly why item SETS, not single items, are needed).

Why this justifies shift vs. reduce decisions:

- If $A \rightarrow \beta_1 \cdot \beta_2$ is valid for $\alpha\beta_1$, and $\beta_2 \neq \epsilon$ (there is still more to match after the dot), this suggests the handle has not fully been shifted onto the stack yet — so the correct move is to shift.
- If $\beta_2 = \epsilon$ (the item is $A \rightarrow \beta_1 \cdot$, fully dotted-through), it looks like $A \rightarrow \beta_1$ IS the handle, and the correct move is to reduce.

Two different valid items can sometimes recommend different actions for the very same viable prefix — this is exactly the source of shift/reduce or reduce/reduce conflicts described above. Some such conflicts can be resolved using one symbol of lookahead (as SLR does); others need the extra power of canonical-LR or LALR; and some grammars simply cannot be resolved by any LR method.

*The central theorem tying all of this together is: **the set of valid items for a viable prefix γ is exactly the set of items reached from the initial state (state 0) by following the path labelled γ through the LR(0) automaton.*** In plain words — the state the automaton is in after reading γ IS the set of all valid items for γ . That is precisely why we are allowed to use the states of the LR(0) automaton to drive shift/reduce decisions in the first place.

6.1 Review Questions

Q: Define a viable prefix.

A: A prefix of a right-sentential form that does not extend past the right end of that sentential form's rightmost handle; equivalently, a string of symbols that could legally sit on the parser's stack at some point in a valid parse.

Q: Define a valid item.

A: An item $A \rightarrow \beta_1 \cdot \beta_2$ is valid for a viable prefix $\alpha\beta_1$ if there is a rightmost derivation $S' \Rightarrow^*_{rm} \alpha A w \Rightarrow^*_{rm} \alpha\beta_1\beta_2w$.

Q: State the property that guarantees a unique grammar symbol per state.

A: Every state of the LR(0) automaton other than the start state 0 has one, unique grammar symbol associated with it, because all transitions into a given state j must be on the same symbol X (though the converse is not guaranteed — two different states may share the same associated symbol).

Q: How do you know an LR(0) automaton state corresponds to a shift action versus a reduce action?

A: If the state's item set contains an item with the dot immediately before a terminal, that terminal triggers a shift; if the item set contains a completed item (dot at the very end), that triggers a reduce for every symbol in the relevant FOLLOW set. A single state can contain both kinds of items for different symbols (no conflict) or, in a non-SLR grammar, for the very same symbol (a conflict).

Q: Viva: Why can't a plain shift-reduce parser (without states) make the same decisions as an LR parser as reliably?

A: Because a plain shift-reduce parser only sees the sequence of grammar symbols on its stack, which does not by itself encode enough information about how those symbols were derived to always resolve shift/reduce ambiguities; the LR parser's states encode exactly the set of valid items for the current viable prefix, giving it the extra context needed.

Q: Viva: What single sentence captures why LR(0) automata can drive shift-reduce decisions at all?

A: Because the state reached after reading a viable prefix γ is exactly the set of all items valid for γ — so the automaton's current state IS the complete, correct summary of what the parser needs to know to decide its next move.

Q: Viva: If two students build the LR(0) automaton for the same grammar but number their states differently, will their final ACTION/GOTO tables represent the same parser?

A: Yes — the underlying item sets, transitions, and hence the parsing decisions are identical; only the arbitrary labels (state numbers) differ, so the tables will look different on paper but define the same parsing behaviour.

7. FIRST and FOLLOW Sets — the Ingredients Behind the Tables

SLR-table construction, panic-mode error recovery, and predictive (LL) parsing all lean on two functions defined purely from the grammar: FIRST and FOLLOW. These two sets are essential background here because SLR's rule (b) needs FOLLOW sets directly.

7.1 Definition of FIRST

$\text{FIRST}(\alpha)$, for any string of grammar symbols α , is the set of terminals that can begin strings derived from α . If $\alpha \Rightarrow^* \epsilon$ (α can derive the empty string), then ϵ is also placed in $\text{FIRST}(\alpha)$.

Rules to compute $\text{FIRST}(X)$ for every grammar symbol X (apply repeatedly until nothing new can be added):

15. If X is a terminal, $\text{FIRST}(X) = \{X\}$.
16. If X is a nonterminal with a production $X \rightarrow Y_1Y_2\cdots Y_k$ ($k \geq 1$): place a in $\text{FIRST}(X)$ whenever $a \in \text{FIRST}(Y_i)$ and $\epsilon \in \text{FIRST}(Y_1), \dots, \text{FIRST}(Y_{i-1})$ (that is, $Y_1\cdots Y_{i-1}$ can all vanish to ϵ , letting Y_i 's first symbol "show through"). If $\epsilon \in \text{FIRST}(Y_j)$ for every $j = 1..k$, add ϵ to $\text{FIRST}(X)$ as well.
17. If $X \rightarrow \epsilon$ is itself a production, add ϵ to $\text{FIRST}(X)$.

To compute FIRST of a whole string $X_1X_2\cdots X_n$: start with the non- ϵ symbols of $\text{FIRST}(X_1)$; if $\epsilon \in \text{FIRST}(X_1)$, also add the non- ϵ symbols of $\text{FIRST}(X_2)$; if ϵ is in both $\text{FIRST}(X_1)$ and $\text{FIRST}(X_2)$, add the non- ϵ symbols of $\text{FIRST}(X_3)$; and so on. Finally add ϵ to the result only if EVERY X_i can vanish to ϵ .

7.2 Definition of FOLLOW

$\text{FOLLOW}(A)$, for a nonterminal A , is the set of terminals a that can appear immediately to the right of A in some right-sentential form — that is, there is some derivation $S \Rightarrow^* \alpha A a \beta$ for some α, β . (There may have been symbols between A and a at some earlier stage of the derivation, but if so, they must have derived ϵ and vanished.) In addition, if A can be the very last symbol of some sentential form, then $\$$ (the end-marker) is included in $\text{FOLLOW}(A)$.

Rules to compute $\text{FOLLOW}(A)$ for every nonterminal A (apply repeatedly until nothing new can be added):

18. Place $\$$ in $\text{FOLLOW}(S)$, where S is the start symbol.
19. If there is a production $A \rightarrow \alpha B \beta$, then everything in $\text{FIRST}(\beta)$ except ϵ is placed in $\text{FOLLOW}(B)$.
20. If there is a production $A \rightarrow \alpha B$, or a production $A \rightarrow \alpha B \beta$ where $\text{FIRST}(\beta)$ contains ϵ , then everything in $\text{FOLLOW}(A)$ is placed in $\text{FOLLOW}(B)$ (because β could vanish, leaving B effectively at the very end, right where A itself would be).

7.3 Worked Example — $\text{Follow}(X) = \text{First}(\beta)$ When $\beta \Rightarrow^* \epsilon$ Implies $\text{Follow}(X) = \text{Follow}(S)$ — Why This Property Holds

This is exactly rule 3 above, restated for a production of the general shape $S \rightarrow \alpha X \beta$ (using S generically for "whatever nonterminal the production belongs to"). The reasoning: whatever terminal can legally follow S in some larger sentential form $S \Rightarrow^* \gamma S \delta \Rightarrow \gamma \alpha X \beta \delta$ can ALSO legally follow X directly, provided β completely disappears ($\beta \Rightarrow^* \epsilon$). When β vanishes, X becomes the last symbol before δ , exactly where S itself would have been — so anything that could follow S can now follow X too. Meanwhile, whatever $\text{FIRST}(\beta)$ contributes (excluding ϵ) obviously can follow X directly regardless of whether β vanishes, since those are the terminals β can literally begin with. Put together: $\text{FOLLOW}(X) \supseteq \text{FIRST}(\beta) \setminus \{\epsilon\}$, and if $\epsilon \in \text{FIRST}(\beta)$, also $\text{FOLLOW}(X) \supseteq \text{FOLLOW}(S)$. This is precisely why FOLLOW-set computation needs a fixed-

point (repeat-until-no-change) style algorithm — FOLLOW(S) itself might depend on FOLLOW of some enclosing nonterminal, discovered only in a later pass.

7.4 Fully Worked FIRST/FOLLOW Example — Parenthesised Expressions (Left-Recursion-Eliminated Form)

Grammar (right-recursive form used after eliminating left recursion from the classic $E \rightarrow E+T \mid T, T \rightarrow T^*F \mid F, F \rightarrow (E) \mid \text{id}$ grammar):

- $E \rightarrow T E'$
- $E' \rightarrow + T E' \mid \epsilon$
- $T \rightarrow F T'$
- $T' \rightarrow * F T' \mid \epsilon$
- $F \rightarrow (E) \mid \text{id}$

Set	Value
$\text{FIRST}(F) = \text{FIRST}(T) = \text{FIRST}(E)$	$\{ (, \text{id} \}$
$\text{FIRST}(E')$	$\{ +, \epsilon \}$
$\text{FIRST}(T')$	$\{ *, \epsilon \}$
$\text{FOLLOW}(E) = \text{FOLLOW}(E')$	$\{), \$ \}$
$\text{FOLLOW}(T) = \text{FOLLOW}(T')$	$\{ +,), \$ \}$
$\text{FOLLOW}(F)$	$\{ +, *,), \$ \}$

Reasoning behind each line

- $\text{FIRST}(F)=\text{FIRST}(T)=\text{FIRST}(E) = \{ (, \text{id} \}$: both productions of F start with these two terminals; T has only one production and it starts with F (which never derives ϵ), so $\text{FIRST}(T)$ just copies $\text{FIRST}(F)$; the same argument carries $\text{FIRST}(E)$.
- $\text{FIRST}(E') = \{ +, \epsilon \}$: one production of E' starts with the terminal $+$, the other production is ϵ — so ϵ is placed directly into $\text{FIRST}(E')$ by rule 3.
- $\text{FIRST}(T') = \{ *, \epsilon \}$: identical reasoning to E' .
- $\text{FOLLOW}(E) = \text{FOLLOW}(E') = \{), \$ \}$: E is the start symbol, so $\$ \in \text{FOLLOW}(E)$ by rule 1. The production $F \rightarrow (E)$ explains why $) \in \text{FOLLOW}(E)$. Since E' only ever appears at the very end of an E-production's body, $\text{FOLLOW}(E')$ must equal $\text{FOLLOW}(E)$.
- $\text{FOLLOW}(T) = \text{FOLLOW}(T') = \{ +,), \$ \}$: T appears in bodies only immediately followed by E' . Since $\text{FIRST}(E')$ (excluding ϵ) = $\{ + \}$, that explains the $+$ in $\text{FOLLOW}(T)$. But $\text{FIRST}(E')$ also contains ϵ , meaning E' can vanish — so everything in $\text{FOLLOW}(E)$ (i.e. $)$ and $\$$) also belongs in $\text{FOLLOW}(T)$. Since T' only ever appears at the end of a T-production, $\text{FOLLOW}(T') = \text{FOLLOW}(T)$.
- $\text{FOLLOW}(F) = \{ +, *,), \$ \}$: identical reasoning to T, since F is always immediately followed by T' in the productions for T.

Tip: For simulation/exam questions, always compute *FIRST* sets completely before starting *FOLLOW* sets, since *FOLLOW*'s rule 3 needs to know whether a suffix β can vanish to ϵ — information that lives in *FIRST*(β).

7.5 Review Questions

Q: Define FIRST(α).

A: The set of terminals that begin strings derivable from α ; if α can derive ϵ , then ϵ is also included in *FIRST*(α).

Q: Define FOLLOW(A).

A: The set of terminals that can appear immediately to the right of A in some right-sentential form; $\$$ is included if A can be the rightmost symbol of some sentential form.

8. Using Ambiguous Grammars — the Dangling-Else Trace

Sometimes it is actually convenient to feed the parser generator a grammar that is technically ambiguous (such as the classic if-then-else grammar), and then resolve the ambiguity by hand-picking which action to keep whenever a conflict appears in the table, instead of rewriting the grammar into a bulkier unambiguous form. The trace below follows the parse of the input *iaea* (a shorthand where *i* = if, *a* = a matched/complete statement, *e* = else) using such a table.

Sentential Form So Far	Action
i	shift
ii	shift
ia	shift
ia	reduce by $S \rightarrow a$
iis	shift
iise	shift
iisea	reduce by $S \rightarrow a$
iiseS	reduce by $S \rightarrow iSeS$
iS	reduce by $S \rightarrow iS$
S	accept

The key decision point is line (5): with state 4 on top of the stack and input symbol *e* next, the parser chooses to SHIFT (not reduce) — this is the conventional resolution that binds each else to the closest unmatched if. Later, at line (9), the same state 4 instead chooses to REDUCE by $S \rightarrow iS$, this time because the lookahead is $\$$ (end of input), so there is no else left to attach to. This demonstrates that the very same state can correctly make two different decisions depending on the lookahead symbol — exactly the mechanism that lets a hand-resolved ambiguous grammar work correctly in practice.

9. Error Recovery in LR Parsing

9.1 When and How Errors Are Detected

An LR parser detects a syntax error precisely when it consults the ACTION table and finds a blank (error) entry. Errors are never detected by consulting the GOTO table — GOTO is only ever consulted right after a reduction has already succeeded, at which point the parser already knows the nonterminal is legal there.

An LR parser announces an error as soon as there is no valid continuation for the portion of input read so far — it never shifts an erroneous symbol onto the stack, and it never waits and reads further input than necessary before flagging the problem.

- A canonical LR parser will not make even a single reduction before announcing an error — the very first invalid lookahead is caught immediately.
- SLR and LALR parsers MAY make several reductions before announcing an error (because they use less precise, merged lookahead information), but critically they will still never shift an erroneous input symbol onto the stack — the error is always caught no later than the point where a shift would have been required.

9.2 Panic-Mode Error Recovery

Panic-mode recovery works as follows:

21. Scan down the stack until a state s is found that has a GOTO defined on some particular nonterminal A .
22. Discard zero or more input symbols until a symbol a is found that can legitimately follow A (that is, $a \in \text{FOLLOW}(A)$).
23. Push the state $\text{GOTO}(s, A)$ onto the stack and resume normal parsing from there.

There may be more than one valid choice of nonterminal A to recover on; typically these are nonterminals that represent whole program constructs — an expression, a statement, or a block. For example, if A is the nonterminal `stmt`, a reasonable resynchronising symbol a might be a semicolon or a closing brace `}`, since those mark the end of a statement sequence in many languages.

What this recovery is really doing: it eliminates the phrase that contains the syntax error. The parser has effectively decided "a string derivable from A contains an error here." Part of that string has already been processed (its result sits as a sequence of states already on the stack); the remainder is still unread in the input. By discarding states from the stack, skipping symbols in the input, and finally pushing $\text{GOTO}(s, A)$, the parser pretends it has found a complete, valid instance of A , and simply carries on parsing as if nothing went wrong.

Tip: Panic mode is deliberately "lossy" — it throws away the specific details of the malformed construct in exchange for being able to keep parsing and (hopefully) find any later, unrelated errors too.

9.3 Phrase-Level Recovery

Phrase-level recovery is more refined: it examines each individual error entry in the LR parsing table and, based on typical language usage, decides on the single most likely programmer mistake that would produce that particular error. An appropriate, specific recovery procedure is then built for that entry — this may modify the top of the stack and/or the first input symbols in whatever way is deemed appropriate for that entry (inserting a missing symbol, deleting an unexpected one, or transposing symbols).

Constraints when designing these routines:

- The parser must never be allowed to enter an infinite loop — at least one input symbol must eventually be removed or shifted, or the stack must eventually shrink once the end of input has been reached.
- Popping a stack state that covers a nonterminal should be avoided wherever possible, because doing so throws away a construct that had already been successfully parsed.

9.4 Error Table for the Ambiguous Expression Grammar — A Worked Example

Consider once more $E \rightarrow E+E \mid E^*E \mid (E) \mid \text{id}$. The table below shows the same LR table modified for error detection and recovery: every state that would previously call for a particular reduction on some inputs has had its blank (error) entries in that row replaced by the reduction itself — this postpones error detection until at least one reduction has happened, but the error is still guaranteed to be caught before any shift move takes place. The remaining blank entries are replaced by calls to numbered error routines e1–e4.

State	id	+	*	(	)	\$	GOTO E
0	s3	e1	e1	s2	e2	e1	1
1	e3	s4	s5	e3	e2	acc	
2	s3	e1	e1	s2	e2	e1	6
3	r4	r4	r4	r4	r4	r4	
4	s3	e1	e1	s2	e2	e1	7
5	s3	e1	e1	s2	e2	e1	8
6	e3	s4	s5	e3	s9	e4	
7	r1	r1	s5	r1	r1	r1	
8	r2	r2	r2	r2	r2	r2	
9	r3	r3	r3	r3	r3	r3	

The four error routines

- **e1** — called from states 0, 2, 4, and 5, all of which expect the beginning of an operand (either id or a left parenthesis), but instead find +, *, or end of input. Recovery: push state 3 (the GOTO of states 0, 2, 4, 5 on id), and issue the diagnostic "missing operand."

- **e2** — called from states 0, 1, 2, 4, and 5 when a right parenthesis is found where it makes no sense. Recovery: remove the right parenthesis from the input, and issue "unbalanced right parenthesis."
- **e3** — called from states 1 or 6 when an operator was expected, but an id or right parenthesis is found instead. Recovery: push state 4 (corresponding to symbol +) onto the stack, and issue "missing operator."
- **e4** — called from state 6 when the end of input is found unexpectedly. Recovery: push state 9 (for a right parenthesis) onto the stack, and issue "missing right parenthesis."

On the erroneous input `id + )`, the parser would invoke `e2` (removing the stray right parenthesis) — demonstrating how these routines let the parser keep going instead of simply halting at the first problem.

9.5 Review Questions

Q: Define panic-mode error recovery.

A: A recovery strategy that discards stack states and input symbols until a nonterminal `A` with a defined `GOTO` is exposed and a legally-following symbol is found, then resumes parsing by pushing `GOTO(s,A)` as if `A` had been matched.

Q: Define phrase-level error recovery.

A: A recovery strategy where each individual blank entry in the parsing table is filled with a call to a specific, hand-designed error routine chosen based on the most likely real-world programmer mistake that produces that particular error.

Q: State the safety condition every error-recovery routine must satisfy.

A: It must guarantee that the parser cannot loop forever: at least one input symbol will eventually be shifted or discarded, or the stack will eventually shrink once input is exhausted.

Q: Why must an error-recovery routine guarantee that at least one input symbol will eventually be shifted or removed?

A: To avoid an infinite loop — if the recovery routine kept the input pointer and stack in a state that immediately re-triggers the very same error rule, the parser would spin forever without ever making progress.

Q: Why is popping a stack state that covers a nonterminal considered bad practice during error recovery?

A: Because doing so throws away a chunk of the input that had already been successfully and correctly parsed, discarding useful, correct partial results purely for the convenience of the recovery routine.

Q: When does panic-mode error recovery push a new state onto the stack?

A: Only after it has scanned down to a state `s` with a defined `GOTO` on some nonterminal `A`, and skipped input symbols until one that legitimately follows `A` is found — at that point it pushes `GOTO(s, A)`.

Q: Differentiate panic-mode and phrase-level error recovery.

A: Panic mode is a single generic mechanism (skip input until something in `FOLLOW(A)` appears, discard the offending phrase) applied uniformly. Phrase-level recovery instead custom-designs a specific fix for

every individual error-table entry, based on the most probable real mistake, giving more precise diagnostics at the cost of much more design effort.

Q: What is a limitation of panic-mode recovery?

A: It can silently discard a large, potentially still-meaningful chunk of the program (everything between the error point and the next synchronising symbol), and may occasionally report a cascade of spurious follow-on errors once the discarded region turns out to matter.

Q: Scenario: while simulating, you find ACTION[state, symbol] is blank. What do you do?

A: Stop and report a syntax error immediately; invoke the appropriate error-recovery routine (panic-mode or phrase-level) rather than guessing an action, since blank cells are the table's built-in definition of invalid input.

Q: Scenario: after a reduce, you look up GOTO but the cell is blank. What does this mean?

A: This should never legitimately happen in a correctly-built table for a valid parse — it usually signals that an earlier pop count was wrong (you popped either too many or too few states), corrupting which state is actually exposed. Re-check the number of symbols popped for the previous reduce.

Q: Explain how panic-mode error recovery interacts with FOLLOW sets.

A: Panic-mode recovery's second step — 'discard symbols until one is found that can legitimately follow A' — literally means 'discard symbols until the current input symbol lies in FOLLOW(A).' This is precisely why FOLLOW sets are described as usable 'synchronizing tokens' during error recovery: they were originally computed for entirely different purposes (SLR-table construction, or LL parsing decisions), but the same sets tell the recovery routine exactly which symbols are safe re-entry points once it decides to pretend that a legal instance of A has just been matched.

Q: Creative/theory: If you were designing a brand-new nonterminal for 'block of statements terminated by end', which FOLLOW-set considerations would you keep in mind to make panic-mode recovery effective for it?

A: You would want FOLLOW of that nonterminal to include clearly-recognisable, low-ambiguity synchronising terminals — keywords like end, or strong punctuation like a semicolon — rather than symbols that are themselves the FIRST of many other constructs (like id), because a synchronising symbol that is too 'common' will cause the recovery routine to resume parsing at a spot that still doesn't correspond to a sensible restart point, potentially triggering a cascade of further spurious errors.

10. The Mechanics of Every Single Move — A Slow-Motion Simulation of $\text{id} * \text{id}$

This section walks through the exact same running-example grammar and table used earlier, but on the shorter string $\text{id} * \text{id}$, going one micro-action at a time — pushing, popping, and looking-up — the way a simulation exam expects you to show your work.

10.1 The Universal Recipe

Every single line of a simulation trace is produced by repeating just two possible recipes:

Recipe A — Shift

24. Look at ACTION[top state, current input symbol]. If it says "shift j":

- Push the input symbol onto the Symbol column (just for our own bookkeeping / for building the tree later).
- Push the state number j onto the Stack column.
- Advance the input pointer by one symbol — the just-shifted symbol is now removed from the remaining input.

Recipe B — Reduce

25. Look at ACTION[top state, current input symbol]. If it says "reduce $A \rightarrow \beta$ ":

- Identify $r = |\beta|$, the number of symbols in the production's right-hand side.
- Pop r symbols from the Symbol column, and pop r states from the Stack column (these must exactly match β — this is the built-in "sanity check" of the whole method).
- Push A (the head of the reducing production) onto the Symbol column.
- Look at the state now exposed on top of the stack after popping — call it t . Look up GOTO[t , A] and push that state number onto the Stack column.
- Do NOT advance the input pointer. The current input symbol is examined again on the very next line, now against the newly-exposed top state.

10.2 Full Trace: $id * id$

#	Production
1	$E \rightarrow E + T$
2	$E \rightarrow T$
3	$T \rightarrow T * F$
4	$T \rightarrow F$
5	$F \rightarrow (E)$
6	$F \rightarrow id$

Stack	Symbol	Input	Action
0	\$	id * id \$	shift to 5 (ACTION[0, id] = s5)
0 5	\$ id	* id \$	reduce by $F \rightarrow id$ (ACTION[5, *] = r6)
0 3	\$ F	* id \$	reduce by $T \rightarrow F$ (ACTION[3, *] = r4)
0 2	\$ T	* id \$	shift to 7 (ACTION[2, *] = s7)
0 2 7	\$ T *	id \$	shift to 5 (ACTION[7, id] = s5)
0 2 7 5	\$ T * id	\$	reduce by $F \rightarrow id$ (ACTION[5, \$] = r6)
0 2 7 10	\$ T * F	\$	reduce by $T \rightarrow T * F$ (ACTION[10, \$] = r3)

Stack	Symbol	Input	Action
0 2	\$ T	\$	reduce by $E \rightarrow T$ (ACTION[2, \$] = r2)
0 1	\$ E	\$	accept (ACTION[1, \$] = acc)

Micro-reasoning for each line

- Line 1: stack is just "0", nothing has been shifted yet, so of course we cannot reduce (there is no completed handle on the stack). ACTION[0, id] = s5, so we push 5. The reason it is NOT a reduce: no item in state 0 is dotted at the very end.
- Line 2: top state 5 corresponds only to the completed item $F \rightarrow id$, so no matter what the lookahead is (as long as it is in FOLLOW(F) = {+, *,), \$}), the only legal action is reduce. It cannot be a shift because state 5 has no outgoing transitions at all.
- Line 3: reducing $F \rightarrow id$ popped exactly 1 state (5) and 1 symbol (id), exposing state 0, then GOTO[0, F] = 3. Now at state 3, the only item is $T \rightarrow F$, again a forced reduce.
- Line 4: after reducing $T \rightarrow F$ (pop 1, expose 0, GOTO[0, T]=2), we are at state 2, which contains BOTH $E \rightarrow T$ (a completed item) and $T \rightarrow T * F$ (an incomplete item with * next). This is why state 2 is NOT simply a reduce state — the ACTION table entry for lookahead * is a shift (s7), while the entries for +,), \$ are reduce (r2). This is exactly why FOLLOW(E) mattered when building the table: * is deliberately excluded from ACTION[2, *]=reduce, because * $\notin$ FOLLOW(E) (well — more precisely, because the shift item $T \rightarrow T * F$ wins the assignment for that particular terminal).
- Line 5–7: a nested repeat of the same $F \rightarrow id$, $T \rightarrow F$ pattern, but this time exposing state 7 (not state 0) underneath, so GOTO[7, F] = 10 (not 3!). The subsequent reduce is $T \rightarrow T * F$: since $T * F$ has 3 symbols, we pop exactly 3 states — 10, 7, and 2 (representing F, *, and T) — exposing state 0, then GOTO[0, T] = 2 is pushed, landing back on state 0's GOTO for T, which is 2 again.
- Line 8: state 2 again, but now lookahead is \$, so ACTION[2, \$] = r2 (reduce $E \rightarrow T$) — this time it genuinely is the reduce entry, not the shift one, purely because the lookahead differs from line 4.
- Line 9: state 1 holds only $E' \rightarrow E$, so ACTION[1, \$] = accept.

10.3 Common Mistakes Students Make in Simulation Exams

- Popping the wrong number of symbols during a reduce — always count $|\beta|$ exactly; for $T \rightarrow T * F$ that is 3, not 1.
- Advancing the input pointer on a reduce — a reduce move NEVER consumes input; only shift does.
- Looking up GOTO using the state that is being popped, instead of the state exposed AFTER popping — GOTO is always taken with respect to the state now on top, after the pop.
- Confusing the ACTION and GOTO halves of the table — terminals (and \$) are looked up only in ACTION; nonterminals are looked up only in GOTO, and only immediately after a reduce.
- Forgetting that state 0 is never removed from the stack — if you ever pop it away, you have made an error somewhere earlier, because a correct parse always leaves state 0 at the very bottom until the final accept.

- Writing "reduce $A \rightarrow \beta$ " but forgetting to also write down which production number it is — most marking schemes want the production number (r_j) explicitly, not just the rule spelled out.
- Mixing up which grammar symbol a state corresponds to when a state is reached by two different symbols (recall Section 1.3 — this is legal and not an error, but it can be confusing when hand-tracing, so always re-derive the symbol from the item set label of that state rather than assuming).

10.4 Tips to Solve Simulation Questions Quickly and Correctly

- Draw two running lists side by side as you go: the Stack (states) and the Symbol (grammar symbols) — never trust yourself to keep both in your head.
- Before every move, physically look only at (a) the state number written at the very top/right end of the Stack column and (b) the current, unconsumed input symbol. Never look further down the stack unless you are in the middle of counting off a reduce's pops.
- When reducing, write out the popped states/symbols explicitly, one by one, crossing them off, before writing the new pushed state/symbol — this prevents off-by-one pop-count errors, which are the single most common source of lost marks.
- Memorise the production numbering table BEFORE starting the trace, and refer back to it by number as you write r_j , so your final trace is self-contained and gradeable without your having to re-derive which rule r_3 or r_6 refers to.
- If ACTION[state, symbol] is blank, stop immediately and report a syntax error — do not try to guess a "reasonable" action; the table's blank entries ARE the definition of an invalid input, by design (Section 9.1).
- Double-check acceptance: the parser should accept only when the stack is exactly "0, state-for-augmented-start-symbol" and the remaining input is just $\$$. If you reach $\$$ but the stack does not look like this, you have made an earlier mistake — keep reducing (there is always at least one more legal reduce available at that point in a correct trace).

10.5 Review Questions

Q: The stack of an SLR parser stores...

A: states of the LR(0) automaton — not raw grammar symbols.

Q: A reduce move changes the input pointer by...

A: zero — reduce never consumes input.

Q: ACTION[i,\$] = accept is set for the item...

A: $[S' \rightarrow S \cdot]$, the completed item for the augmented start production.

Q: FOLLOW sets are used in SLR construction to decide...

A: which lookahead symbols make a completed item's reduce action legal (rule b).

Q: A blank ACTION cell means...

A: a syntax error has been detected at that (state, symbol) pair.

Q: If the same ACTION cell wants both shift and reduce, this is called a...

A: shift/reduce conflict, and the grammar is not SLR(1).

Q: The number of states popped for a reduce $A \rightarrow \beta$ equals...

A: $|\beta|$, the number of grammar symbols in the production body (0 for $A \rightarrow \epsilon$).

Q: GOTO is only ever consulted...

A: immediately after a reduce move, using the nonterminal just produced.

Q: Every SLR(1) grammar is...

A: unambiguous (but not every unambiguous grammar is SLR(1)).

Q: Canonical LR parsers detect errors...

A: as soon as possible — never making even a single reduction before reporting an error, unlike SLR/LALR which may reduce first.

11. Fully Worked Practice Problems

These are complete, from-scratch solutions to a set of practice questions. Work through each one with pen and paper before checking the solution.

11.1 Problem A — Completing an Incomplete SLR(1) Table (if / while / a grammar)

Grammar:

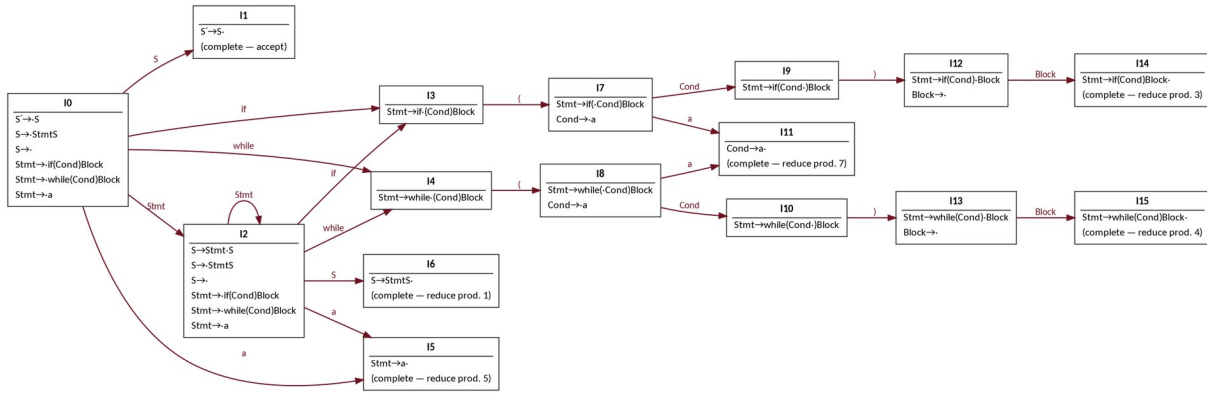
- 1. $S \rightarrow \text{Stmt } S$
- 2. $S \rightarrow \epsilon$
- 3. $\text{Stmt} \rightarrow \text{if } (\text{Cond}) \text{Block}$
- 4. $\text{Stmt} \rightarrow \text{while } (\text{Cond}) \text{Block}$
- 5. $\text{Stmt} \rightarrow a$
- 6. $\text{Block} \rightarrow \epsilon$
- 7. $\text{Cond} \rightarrow a$

$\text{FIRST}(S) = \{\text{if}, \text{while}, a, \epsilon\}$, $\text{FIRST}(\text{Stmt}) = \{\text{if}, \text{while}, a\}$, $\text{FIRST}(\text{Block}) = \{\epsilon\}$, $\text{FIRST}(\text{Cond}) = \{a\}$

$\text{FOLLOW}(S) = \{\$, \}$, $\text{FOLLOW}(\text{Stmt}) = \{\text{if}, \text{while}, a, \$\}$, $\text{FOLLOW}(\text{Block}) = \{\text{if}, \text{while}, a, \$\}$, $\text{FOLLOW}(\text{Cond}) = \{\}$

Question: Using the provided context-free grammar (CFG), LR(0) automaton, and the FIRST and FOLLOW sets, first verify whether the given SLR(1) parsing table is complete. After completing the table (if necessary), demonstrate whether the input string is grammatically correct or not. Input string: if (a) while (a) a. (Note: the table must be drawn out in full.)

LR(0) Automaton (as provided, 16 states, I0–I15):



SLR(1) Parse Table (as given — note the blank cells):

State	while	if	(	a	)	\$	GOTO S, Stmt, Block, Cond
0		S3		S5			S=1
1							
2		S3		S5			Stmt=2
3			S7				
4			S8				
5	R5	R5				R5	
6						R1	
7				S11			Cond=9
8				S11			Cond=10
9					S12		
10					S13		
11					R7		
12							Block=14
13							Block=15
14	R3			R3		R3	
15	R4			R4		R4	

Solution:

Step 1 — Auditing the given table against the automaton

Applying the SLR reduce-action rule (rule (b)) to every completed item in every state (I0 has $S \rightarrow \cdot$, I2 has $S \rightarrow \cdot$, I6 has $S \rightarrow StmtS \cdot$, I5 has $Stmt \rightarrow a \cdot$, I11 has $Cond \rightarrow a \cdot$, I12/I13 have $Block \rightarrow \cdot$, I14/I15 have $Stmt \rightarrow (...)Block \cdot$) turns up several entries missing from the table as originally given:

- State 0: missing ACTION[0, while] = shift 4 (item $Stmt \rightarrow \cdot while(Cond)Block$), missing ACTION[0, \$] = reduce by $S \rightarrow \epsilon$ (production 2, since $\$ \in FOLLOW(S)$), and missing GOTO[0, Stmt] = 2.
- State 1: missing ACTION[1, \$] = accept (item $S' \rightarrow S \cdot$ must always accept on \$).

- State 2: missing ACTION[2, while] = shift 4, missing ACTION[2, \$] = reduce by $S \rightarrow \epsilon$, and missing GOTO[2, S] = 6.
- State 5: missing ACTION[5, a] = reduce by $\text{Stmt} \rightarrow a$ (production 5), since $a \in \text{FOLLOW}(\text{Stmt})$.
- States 12 and 13: entirely missing the reduce-by-Block $\rightarrow\epsilon$ action (production 6) on if, while, a, and \$ — every one of these belongs in $\text{FOLLOW}(\text{Block})=\{\text{if,while,a,\$}\}$.
- State 14: missing ACTION[14, if] = reduce by $\text{Stmt} \rightarrow \text{if}(\text{Cond})\text{Block}$ (production 3), since if $\in \text{FOLLOW}(\text{Stmt})$.
- State 15: missing ACTION[15, if] = reduce by $\text{Stmt} \rightarrow \text{while}(\text{Cond})\text{Block}$ (production 4), likewise.

Step 2 — The corrected, complete table

State	while	if	(	a	)	\$	GOTO S,Stmt,Block,Cond
0	S4	S3		S5		R2	S=1, Stmt=2
1						accept	
2	S4	S3		S5		R2	S=6, Stmt=2
3			S7				
4			S8				
5	R5	R5		R5		R5	
6						R1	
7				S11			Cond=9
8				S11			Cond=10
9					S12		
10					S13		
11					R7		
12	R6	R6		R6		R6	Block=14
13	R6	R6		R6		R6	Block=15
14	R3	R3		R3		R3	
15	R4	R4		R4		R4	

Tip: Every reduce row's set of columns should exactly equal the FOLLOW set of the nonterminal being reduced — this is the fastest way to check your own completed table before you start simulating: count the non-blank cells in each reduce row and compare against the FOLLOW-set size.

Step 3 — Simulating if (a) while (a) a

Stack	Symbol	Input	Action
0	\$	if (a) while (a) a \$	shift 3 — ACTION[0,if]=S3

Stack	Symbol	Input	Action
0 3	\$ if	(a) while (a) a \$	shift 7 — ACTION[3,(]=S7
0 3 7	\$ if (	a) while (a) a \$	shift 11 — ACTION[7,a]=S11
0 3 7 11	\$ if (a	) while (a) a \$	reduce Cond→a — pop 1, GOTO[7,Cond]=9
0 3 7 9	\$ if (Cond	) while (a) a \$	shift 12 — ACTION[9,)=S12
0 3 7 9 12	\$ if (Cond)	while (a) a \$	reduce Block→ε — pop 0, GOTO[12,Block]=14
0 3 7 9 12 14	\$ if (Cond) Block	while (a) a \$	reduce Stmt→if(Cond)Block — pop 5, GOTO[0,Stmt]=2
0 2	\$ Stmt	while (a) a \$	shift 4 — ACTION[2,while]=S4
0 2 4	\$ Stmt while	(a) a \$	shift 8 — ACTION[4,(]=S8
0 2 4 8	\$ Stmt while (	a) a \$	shift 11 — ACTION[8,a]=S11
0 2 4 8 11	\$ Stmt while (a	) a \$	reduce Cond→a — pop 1, GOTO[8,Cond]=10
0 2 4 8 10	\$ Stmt while (Cond	) a \$	shift 13 — ACTION[10,]=S13
0 2 4 8 10 13	\$ Stmt while (Cond)	a \$	reduce Block→ε — pop 0, GOTO[13,Block]=15
0 2 4 8 10 13 15	\$ Stmt while (Cond) Block	a \$	reduce Stmt→while(Cond)Block — pop 5, GOTO[2,Stmt]=2
0 2 2	\$ Stmt Stmt	a \$	shift 5 — ACTION[2,a]=S5
0 2 2 5	\$ Stmt Stmt a	\$	reduce Stmt→a — pop 1, GOTO[2,Stmt]=2
0 2 2 2	\$ Stmt Stmt Stmt	\$	reduce S→ε — pop 0, GOTO[2,S]=6
0 2 2 2 6	\$ Stmt Stmt Stmt S	\$	reduce S→StmtS — pop 2, GOTO[2,S]=6
0 2 2 6	\$ Stmt Stmt S	\$	reduce S→StmtS — pop 2, GOTO[2,S]=6
0 2 6	\$ Stmt S	\$	reduce S→StmtS — pop 2, GOTO[0,S]=1
0 1	\$ S	\$	accept — ACTION[1,\$]=accept

Result: the string if (a) while (a) a is grammatically CORRECT. Notice how the grammar treats it as three consecutive statements (an if-statement, a while-statement, and a plain a) followed by the empty S at the very end — this is why several $S \rightarrow \epsilon$ and $S \rightarrow \text{Stmt } S$ reductions unwind one after another right before acceptance; that unwinding is completely normal for a right-recursive "list of statements" grammar and is not a sign of an error.

11.2 Problem B — Finding Conflicts in a Statement-List / Assignment Grammar

Grammar:

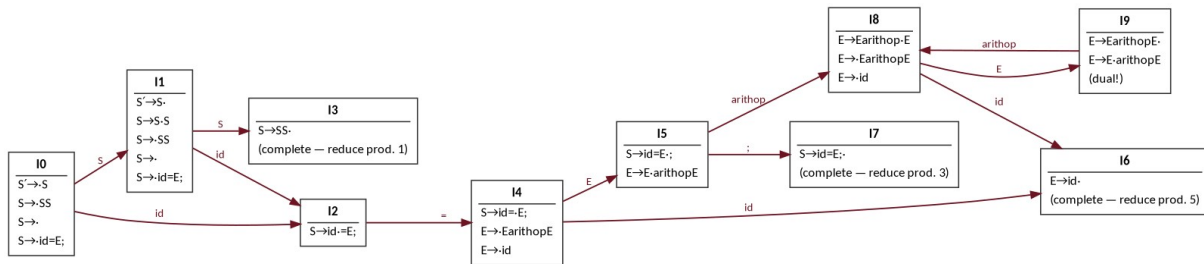
- 1. $S \rightarrow S S$
- 2. $S \rightarrow \epsilon$
- 3. $S \rightarrow \text{id} = E ;$
- 4. $E \rightarrow E \text{ arithop } E$
- 5. $E \rightarrow \text{id}$

Question: Given the following grammar, produce the LR(0) automaton. Terminals: {id, =, ;, arithop}. For the grammar above, FOLLOW(S) = {\$, id} and FOLLOW(E) = {;, arithop}. Now, by showing the SLR parsing table, determine the conflicting actions (if any).

Solution:

Building the LR(0) automaton (item sets)

LR(0) automaton for the assignment/expression grammar (10 states, I0–I9).



Where the conflicts appear

State I0 contains BOTH the completed item $S \rightarrow \cdot$ (asking for a reduce by production 2, $S \rightarrow \epsilon$, on every symbol in FOLLOW(S)={\$, id}) AND the item $S \rightarrow \text{id} = E ;$ (asking for a shift on id, since id begins that production). Since id belongs to both sets, ACTION[I0, id] is asked to be simultaneously **shift** (into I2) and **reduce** ($S \rightarrow \epsilon$). This is a **shift/reduce conflict**. The exact same conflict re-appears in state I1, which contains the identical pair of items.

State I9 contains BOTH the completed item $E \rightarrow \text{arithop} E \cdot$ (asking for a reduce by production 4 on every symbol in FOLLOW(E)={;, arithop}) AND the incomplete item $E \rightarrow \text{arithop} E \cdot$ (asking for a shift on arithop, since arithop follows the dot). Since arithop belongs to both sets, ACTION[I9, arithop] is asked to be both shift (into I8) and reduce (production 4) — a second, independent **shift/reduce conflict**.

Conclusion: this grammar is **NOT SLR(1)** — it has shift/reduce conflicts at (at least) states I0/I1 on id, and at state I9 on arithop. In practice a real compiler would resolve the I9 conflict by choosing SHIFT (which makes arithop left-associative), the same trick used for the classic ambiguous expression grammar; the I0/I1 conflict, however, reflects a genuine structural ambiguity between "is this id the start of a brand-new statement, or should we first close off an empty statement-list here" and needs a grammar rewrite (or a stronger parser generator) to resolve cleanly.

11.3 Problem C — Full SLR(1) Construction for $A \rightarrow B A C \mid w \mid \epsilon$, $B \rightarrow x$, $C \rightarrow y$

Question: Construct the SLR(1) parser for the grammar below. [5 points for proper automaton construction, 3 points for computing FIRST and FOLLOW, 4 points for the ACTION table, and 3 points for the GOTO table] Grammar (alphabet of terminals $\{x, y, w\}$):

- 1. $A \rightarrow B A C$
- 2. $B \rightarrow x$
- 3. $C \rightarrow y$
- 4. $A \rightarrow w$
- 5. $A \rightarrow \epsilon$

Solution:

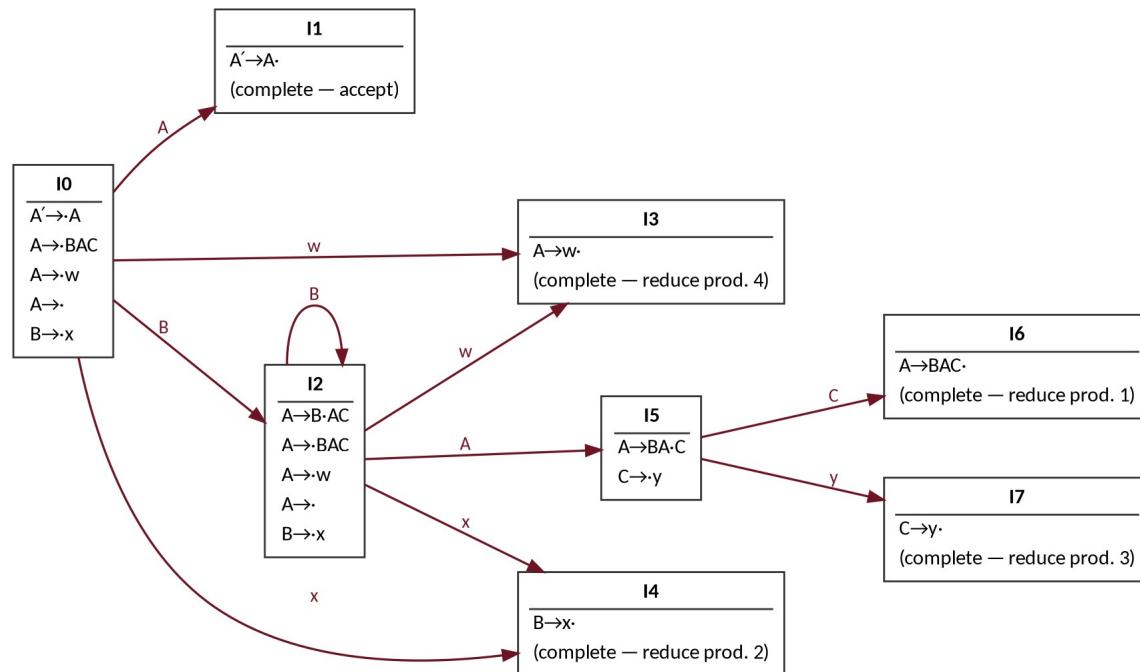
FIRST and FOLLOW

Set	Value
FIRST(B)	{x}
FIRST(C)	{y}
FIRST(A)	{x, w, ϵ }
FOLLOW(A)	{y, \$}
FOLLOW(B)	{x, w, y, \$}
FOLLOW(C)	{y, \$}

- FOLLOW(A) = {y, \$}: $A' \rightarrow A$ gives \$; production $A \rightarrow BAC$ has A immediately followed by C, and FIRST(C)={y} contributes y.
- FOLLOW(B) = {x, w, y, \$}: production $A \rightarrow BAC$ has B immediately followed by A; FIRST(A)={x,w, ϵ } contributes x and w directly, and because $\epsilon \in \text{FIRST}(A)$, everything in FOLLOW(A)={y,\$} is also inherited by FOLLOW(B).
- FOLLOW(C) = {y, \$}: C sits at the very end of production $A \rightarrow BAC$, so FOLLOW(C) simply equals FOLLOW(A).

LR(0) Automaton (8 states)

LR(0) automaton for $A \rightarrow BAC \mid w \mid \epsilon$, $B \rightarrow x$, $C \rightarrow y$ (8 states, I0-I7).



Complete ACTION / GOTO Table — No Conflicts (this grammar IS SLR(1))

State	x	y	w	\$	GOTO A	GOTO B	GOTO C
0	S4	R5	S3	R5	1	2	
1				accept			
2	S4	R5	S3	R5	5	2	
3		R4		R4			
4	R2	R2	R2	R2			
5		S7					6
6		R1		R1			
7		R3		R3			

Because every reduce row's non-blank columns match exactly the FOLLOW set of the nonterminal being reduced, and no cell is ever asked to hold two different actions, this grammar has a clean SLR(1) table with zero conflicts.

11.4 Problem D — Parsing ((id + (id))) with the Standard Expression Table

Question: Show how an SLR(1) parser with the above grammar rules and parsing tables (grammar $E \rightarrow E + T \mid T, T \rightarrow T * F \mid F, F \rightarrow (E) \mid id$) processes the following input string: ((id + (id))).

Solution:

Stack	Symbol	Input (remaining)	Action
0	\$	((id+(id)))\$	shift 4
0 4	\$ (	(id+(id)))\$	shift 4
0 4 4	\$ ((	id+(id)))\$	shift 5
0 4 4 5	\$ ((id	+(id)))\$	reduce $F \rightarrow id$, GOTO[4,F]=3
0 4 4 3	\$ ((F	+(id)))\$	reduce $T \rightarrow F$, GOTO[4,T]=2
0 4 4 2	\$ ((T	+(id)))\$	reduce $E \rightarrow T$, GOTO[4,E]=8
0 4 4 8	\$ ((E	+(id)))\$	shift 6
0 4 4 8 6	\$ ((E+	(id)))\$	shift 4
0 4 4 8 6 4	\$ ((E+(	id)))\$	shift 5
0 4 4 8 6 4 5	\$ ((E+(id	)))\$	reduce $F \rightarrow id$, GOTO[4,F]=3
0 4 4 8 6 4 3	\$ ((E+(F	)))\$	reduce $T \rightarrow F$, GOTO[4,T]=2
0 4 4 8 6 4 2	\$ ((E+(T	)))\$	reduce $E \rightarrow T$, GOTO[4,E]=8
0 4 4 8 6 4 8	\$ ((E+(E	)))\$	shift 11
0 4 4 8 6 4 8 11	\$ ((E+(E)	))\$	reduce $F \rightarrow (E)$, pop 3, GOTO[6,F]=3
0 4 4 8 6 3	\$ ((E+F	))\$	reduce $T \rightarrow F$, GOTO[6,T]=9
0 4 4 8 6 9	\$ ((E+T	))\$	reduce $E \rightarrow E+T$, pop 3, GOTO[4,E]=8
0 4 4 8	\$ ((E	))\$	shift 11
0 4 4 8 11	\$ ((E)	)\$	reduce $F \rightarrow (E)$, pop 3, GOTO[4,F]=3
0 4 3	\$ (F	)\$	reduce $T \rightarrow F$, GOTO[4,T]=2
0 4 2	\$ (T	)\$	reduce $E \rightarrow T$, GOTO[4,E]=8
0 4 8	\$ (E	)\$	shift 11
0 4 8 11	\$ (E)	\$	reduce $F \rightarrow (E)$, pop 3, GOTO[0,F]=3
0 3	\$ F	\$	reduce $T \rightarrow F$, GOTO[0,T]=2
0 2	\$ T	\$	reduce $E \rightarrow T$, GOTO[0,E]=1
0 1	\$ E	\$	accept

Result: ((id+(id))) is grammatically CORRECT. This trace is the best practice example for the rule "reduce $F \rightarrow (E)$ always pops exactly 3 states/symbols" (the open-paren, the E, and the close-paren) — a length that trips up many students because it does not equal the number of characters, only the number of grammar symbols in the production body.

11.5 Problem E — while id do end (Nested Statement Grammar)

Grammar:

- 1. $S \rightarrow AS$

- 2. $S \rightarrow \epsilon$
- 3. $A \rightarrow W$
- 4. $A \rightarrow \text{other}$
- 5. $W \rightarrow \text{while } E \text{ do } S \text{ end}$
- 6. $E \rightarrow \text{id}$

Question: Determine by simulating parsing whether the following string is syntactically correct or not according to the parsing table below. The space in the string separates the tokens received from the lexical analyzer. Input String: while id do end. Terminals: {other, while, do, end, id}.

Solution:

Stack	Symbol	Input	Action
0	\$	while id do end \$	shift 5
0 5	\$ while	id do end \$	shift 7
0 5 7	\$ while id	do end \$	reduce $E \rightarrow \text{id}$, GOTO[5,E]=8
0 5 8	\$ while E	do end \$	shift 9
0 5 8 9	\$ while E do	end \$	reduce $S \rightarrow \epsilon$, GOTO[9,S]=10
0 5 8 9 10	\$ while E do S	end \$	shift 11
0 5 8 9 10 11	\$ while E do S end	\$	reduce $W \rightarrow \text{while } E \text{ do } S \text{ end}$, pop 5, GOTO[0,W]=3
0 3	\$ W	\$	reduce $A \rightarrow W$, GOTO[0,A]=2
0 2	\$ A	\$	reduce $S \rightarrow \epsilon$, GOTO[2,S]=6
0 2 6	\$ A S	\$	reduce $S \rightarrow AS$, pop 2, GOTO[0,S]=1
0 1	\$ S	\$	accept

Result: while id do end is grammatically CORRECT.